Project 2

Janardhana Reddy M S
jr6922
N12832723

- How to run the program: Just run the main.py file: python3 main.py
- Parameters being chosen:
  Window size w, used in Harris.py
  For ball image 7, since the features are fairly big, and there are no fine corners.
  For moebius: 5, Since we have lot more fine corners which would have been missed due to large window size.
  For outdoor: 9, since there are a lot of features that might be considered corners in this image, due to the lower part of the image being blurry.

  Threshold (T): used in harris.py to control sensitivity.
  Used to threshold of 0.3 for all the images 1 and 2, 0.01 for image 3 since its noisy.

  Parameter K :
  For image 1: Balls = 0.07, so that it's less sensitive to detect edges, used 0.07 since we have fairly large features, and it detects only corners.
  For image 2: Moebius = 0.04, since we large number of fine features, I have decreased the K value.
  For image 3: Outdoor: 0.03, since we have a large number of fine features, but the image is very noisy.

  Window Size S: Used to set the window size when doing correlation
  For image 1: Since we have large features, I have set this to 9 to get more context/
  For image 2: Since we have fine features, I have set this to 5
  For image 3: Set this to 9, since the blur area is being detected as corners, if the S is low, so setting to a higher value increases the chance of actual corners being recognized and false ones not being recognized.

  Gaussian filter: Yes, I blur the image out and smooth the noise.
  Window Size: 7*7 and SD = 1.

- Source Code: I have 5 separate each of which executes the feature the file is named after,
  Main.py: Run all these files in a pipeline.
  Harris.py: Detect Harris corners
  Matcher.py: Matching corners in left and right images
  Depth.py: Compute relative depth using these matched points.
  Utils.py: Handle image drawing, ASCII output, and file saving.

Main.py:

```
import cv2
import glob
import numpy as np

#I have separate python file for every feature step in our project

from harris import detect_harris_corners
from matcher import match_features
from depth import compute_depth_map, normalize_depth_map, visualize_depth_map
from utils import draw_corners, display_and_save_depth_map, save_corners_to_ascii,save_matches_and_depth,
draw_matches,save_image

#reading both the images
left_images = glob.glob("images/Outdoor_left.*")
right_images = glob.glob("images/Outdoor_right.*")


left_path = left_images[0]
right_path = right_images[0]

if left_path is None or right_path is None:
    raise ValueError("Images not found ")

left_img = cv2.imread(left_path, cv2.IMREAD_GRAYSCALE)
right_img = cv2.imread(right_path, cv2.IMREAD_GRAYSCALE)

#Removing the noise
left_blur = cv2.GaussianBlur(left_img, (5, 5), 1)
right_blur = cv2.GaussianBlur(right_img, (5, 5), 1)

# find corners in both images using harris corner files
left_corners = detect_harris_corners(left_blur)
right_corners = detect_harris_corners(right_blur)


# now try to match the corners from left to right image using correlation
matches, disparities = match_features(left_img, right_img, left_corners, right_corners)
```

```
# show me that its done and save the depth map
print("done. depth map is ready .")


# ASCII outputs
save_corners_to_ascii(left_corners, "left_corners_outdoor.txt")
save_corners_to_ascii(right_corners, "right_corners_outdoor.txt")

# Corner visualization
draw_corners(left_img, left_corners, "left_corners_outdoor.bmp")
draw_corners(right_img, right_corners, "right_corners_outdoor.bmp")

# Depth map
normalized_depth = compute_depth_map(left_img.shape, matches)
display_and_save_depth_map(normalized_depth, filename="relative_depth_map_outdoor.png")
matched_vis = draw_matches(left_img, left_corners, right_img, right_corners, [(x1, y1, x2, y2) for (x1, y1), (x2, y2) in
matches.items()])
save_image("matched_features_outdoor.png", matched_vis)
# Save relative depth map as .bmp
cv2.imwrite("depth_map_outdoor.bmp", normalized_depth)

# Save matches, correlation, and depth info
save_matches_and_depth(matches, disparities, normalized_depth, "matches_and_depth_outdoor.txt")
```

Harris.py

```
import cv2
import numpy as np

def detect_harris_corners(image, window_size=9, k=0.06, threshold_ratio=0.02):
    #Get gradients in x and y directions using Sobel filter
    Ix = cv2.Sobel(image, cv2.CV_64F, 1, 0, ksize=3)  # horizontal edges
    Iy = cv2.Sobel(image, cv2.CV_64F, 0, 1, ksize=3)  # vertical edges

    # We Compute products of derivatives
    Ix2 = Ix ** 2
    Iy2 = Iy ** 2
```

```
    Ixy = Ix * Iy

    #Rather than using a loop to move over picture with a window to sum the total, i am using gaussian blur to make the process faster.
    #Apply Gaussian blur to these derivative products
    Sx2 = cv2.GaussianBlur(Ix2, (window_size, window_size), sigmaX=1)
    Sy2 = cv2.GaussianBlur(Iy2, (window_size, window_size), sigmaX=1)
    Sxy = cv2.GaussianBlur(Ixy, (window_size, window_size), sigmaX=1)

    # Compute Harris response R = det(M) - k * trace(M)^2
    h, w = image.shape
    R = np.zeros((h, w))
    for y in range(h):
        for x in range(w):
            A = Sx2[y, x]
            B = Sxy[y, x]
            C = Sy2[y, x]

            det = A * C - B ** 2
            trace = A + C
            R[y, x] = det - k * (trace ** 2)

    # Compute adaptive threshold based on max R
    threshold = threshold_ratio * np.max(R)

    # Non-maximum suppression to get only distinct corners,
    corners = []
    offset = window_size // 2
    for y in range(offset, h - offset):
        for x in range(offset, w - offset):
            val = R[y, x]
            if val > threshold:
                local_patch = R[y - offset:y + offset + 1, x - offset:x + offset + 1]
                if val == np.max(local_patch):
                    corners.append((x, y))

    return corners
```

Matcher.py

```
import cv2
import numpy as np

def match_features(left_img, right_img, left_keypoints, right_keypoints, window_size=9, row_tolerance=5):
    # This dictionary will store the best match in the right image for each keypoint in the left image.
    matches = {}

    # We will also keep track of the correlation value for each match.
    disparities = {}

    half_window = window_size // 2

    # Loop through each keypoint in the left image
    for (x1, y1) in left_keypoints:
        best_corr = -1     # Best correlation value found so far
        best_match = None  # Best matching point in the right image

        # Check if the patch around this keypoint would go out of bounds. Skip if it does.
        if (y1 - half_window < 0 or y1 + half_window >= left_img.shape[0] or
            x1 - half_window < 0 or x1 + half_window >= left_img.shape[1]):
            continue

        # Extract the patch centered at this keypoint from the left image
        patch_left = left_img[y1 - half_window:y1 + half_window + 1, x1 - half_window:x1 + half_window + 1]
        patch_left = patch_left.astype(np.float32)
        patch_left_mean = np.mean(patch_left)
        patch_left -= patch_left_mean

        # Now look for the best matching patch in the right image
        for (x2, y2) in right_keypoints:
            # Only compare patches on approximately the same row to reduce false matches
            if abs(y2 - y1) > row_tolerance:
                continue

            # Again, check patch boundaries skip if going out of image bounds
            if (y2 - half_window < 0 or y2 + half_window >= right_img.shape[0] or
                x2 - half_window < 0 or x2 + half_window >= right_img.shape[1]):
                continue
```

```
            # Extract the matching patch from the right image
            patch_right = right_img[y2 - half_window:y2 + half_window + 1, x2 - half_window:x2 + half_window + 1]
            patch_right = patch_right.astype(np.float32)
            patch_right_mean = np.mean(patch_right)
            patch_right -= patch_right_mean

            # Compute the normalized cross correlation between the two patches we have
            numerator = np.sum(patch_left * patch_right)
            denominator = np.sqrt(np.sum(patch_left ** 2) * np.sum(patch_right *2))

            if denominator == 0:
                continue

            corr = numerator / denominator

            # If this is the best correlation we have got, then store it
            if corr > best_corr:
                best_corr = corr
                best_match = (x2, y2)

        # Once we check all candidates, save the best match for this keypoint
        if best_match:
            matches[(x1, y1)] = best_match
            disparities[(x1, y1)] = best_corr

    # Return both the matches and their correlation scores
    return matches, disparities
```

Depth.py

```
import numpy as np
import cv2

def compute_depth_map(left_image_shape, matches, k=1):
    # Get the image dimensions
    height, width = left_image_shape[:2]

    # Start with all zero depth map
    depth_map = np.zeros((height, width), dtype=np.uint8)
```

```
# We will store each computed depth 'z' and the corresponding pixel position
z_values = []
positions = []

# Loop through all the matching feature points
for (x1, y1), (x2, y2) in matches.items():
    # Disparity is the difference in x-coordinates between the left and right points
    disparity = abs(x1 - x2)
    if disparity == 0:
        continue  # Skip if disparity is zero to avoid dividing by zero

    # Compute relative depth using the simple inverse formula z = k / disparity
    z = k / disparity
    z_values.append(z)
    positions.append((int(y1), int(x1)))  # Save (i, j) position for this depth

# If no valid depth values were computed, just return the blank map
if len(z_values) == 0:
    return depth_map

# Convert depth values into a NumPy array to easily scale them
z_values = np.array(z_values)
z_min, z_max = np.min(z_values), np.max(z_values)

# Prevent division by zero if all depths are the same
if z_max == z_min:
    z_max += 1e-6

# Loop through all computed z-values and map them to grayscale intensity
for idx, z in enumerate(z_values):
    # Normalize the depth to the range 10–255 (darker = closer)
    z_scaled = 255 - int(245 * ((z - z_min) / (z_max - z_min) + 0.5))
    z_scaled = np.clip(z_scaled, 10, 255)

    # Place the scaled depth value in the correct pixel location
    y, x = positions[idx]
    depth_map[y, x] = z_scaled

return depth_map
```

```
def normalize_depth_map(depth_map):
    # Extract only the valid (not zero) depth values
    valid_depths = depth_map[depth_map > 0]
    if len(valid_depths) == 0:
        return np.zeros_like(depth_map, dtype=np.uint8)

    # Find the min and max of the valid depth range
    z_min, z_max = valid_depths.min(), valid_depths.max()
    if z_max == z_min:
        z_max += 1  # Prevent divide-by-zero

    # Create a blank image to hold normalized depth values
    normalized = np.zeros_like(depth_map, dtype=np.uint8)

    # Scale all valid depth values to a 10–255 grayscale range
    scaled = 255 - ((245 * (depth_map - z_min) / (z_max - z_min)) + 0.5).astype(np.uint8)
    normalized[depth_map > 0] = scaled[depth_map > 0]

    return normalized
```

Utils.py

```
import cv2
import numpy as np

def draw_keypoints(image, keypoints, color=(0, 255, 0)):
    # This function draws small circles on the image at the given keypoints, useful to visually verify if corners or other points were detected correctly.

    img_with_kp = image.copy()

    # If it's a grayscale image, convert to color so we can draw in color.
    if len(img_with_kp.shape) == 2:
        img_with_kp = cv2.cvtColor(img_with_kp, cv2.COLOR_GRAY2BGR)

    # Draw a small circle at each keypoint (x, y).
```

```
    for x, y in keypoints:
        cv2.circle(img_with_kp, (int(x), int(y)), 3, color, -1)

    return img_with_kp


def draw_matches(img1, kp1, img2, kp2, matches):
    # This function creates a side-by-side visualization of matching keypoints between two images.

    # Create a blank canvas wide enough to place both images side by side.
    h = max(img1.shape[0], img2.shape[0])
    w = img1.shape[1] + img2.shape[1]

    # Convert grayscale images to color so we can draw colorful lines and points.
    if len(img1.shape) == 2:
        img1 = cv2.cvtColor(img1, cv2.COLOR_GRAY2BGR)
    if len(img2.shape) == 2:
        img2 = cv2.cvtColor(img2, cv2.COLOR_GRAY2BGR)

    combined = np.zeros((h, w, 3), dtype=np.uint8)
    combined[:img1.shape[0], :img1.shape[1]] = img1
    combined[:img2.shape[0], img1.shape[1]:] = img2

    # Draw lines between matched keypoints
    for (x1, y1, x2, y2) in matches:
        pt1 = (int(x1), int(y1))
        pt2 = (int(x2 + img1.shape[1]), int(y2))  # Adjust x2 for right image's offset

        cv2.circle(combined, pt1, 4, (0, 255, 0), -1)  # Green for left point
        cv2.circle(combined, pt2, 4, (0, 0, 255), -1)  # Red for right point
        cv2.line(combined, pt1, pt2, (255, 0, 0), 1)   # Blue line connecting them

    return combined


def save_image(path, image):
    # Just a simple wrapper to save an image using OpenCV
    cv2.imwrite(path, image)
```

```
def normalize_image(img):
    # Normalizes an image so that its values span the full range [0, 255]
    # This is helpful for display or saving when your image has float or narrow value ranges
    norm = cv2.normalize(img, None, 0, 255, cv2.NORM_MINMAX)
    return norm.astype(np.uint8)


def draw_corners(image, corners, save_path):
    # Draws a red '+' marker at each corner and saves the image to disk
    img_with_corners = cv2.cvtColor(image.copy(), cv2.COLOR_GRAY2BGR)
    for (x, y) in corners:
        cv2.drawMarker(img_with_corners, (x, y), (0, 0, 255), markerType=cv2.MARKER_CROSS, markerSize=10,
thickness=1)
    cv2.imwrite(save_path, img_with_corners)


def display_and_save_depth_map(depth_map, filename="outputs/depth_map.png"):
    cv2.imwrite(filename, depth_map)


def save_corners_to_ascii(corners, filename):
    # Saves Harris corners to a .txt file in the format:
    # First line = number of corners
    # Next lines = (i, j) = (row, column) for each corner, sorted top-to-bottom and left-to-right

    corners = sorted(corners, key=lambda pt: (pt[0], pt[1]))  # Sort by y, then x
    with open(filename, 'w') as f:
        f.write(f"{len(corners)}\n")
        for x, y in corners:
            f.write(f"{y} {x}\n")


def save_matches_and_depth(matches, disparities, depth_map, filename):
    # Saves matching feature pairs and their depth/correlation info to a .txt file.
    # Format:
    # First line: number of matches
    # -Each line: i_left j_left i_right j_right correlation depth

    with open(filename, 'w') as f:
        f.write(f"{len(matches)}\n")
        for ((x1, y1), (x2, y2)) in matches.items():
```

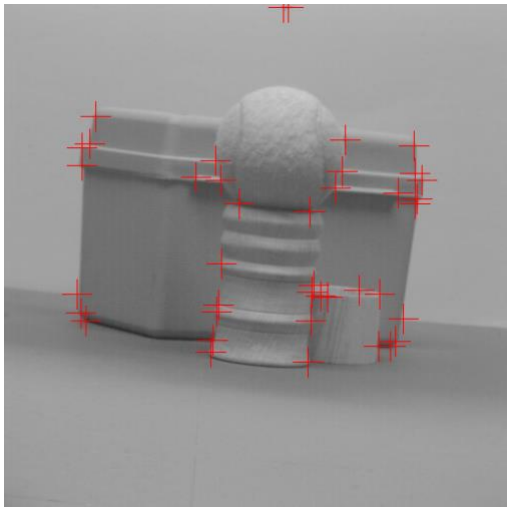
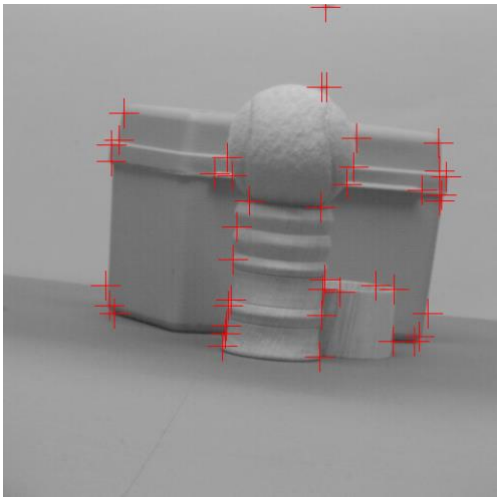
```
    disparity = x1 - x2
    if disparity == 0:
        continue
    z = 1 / disparity
    r = disparities.get((x1, y1), 0.0)
    z_scaled = int(depth_map[y1, x1])
    f.write(f"{y1} {x1} {y2} {x2} {r:.4f} {z_scaled}\n")
```

- **Output Images**

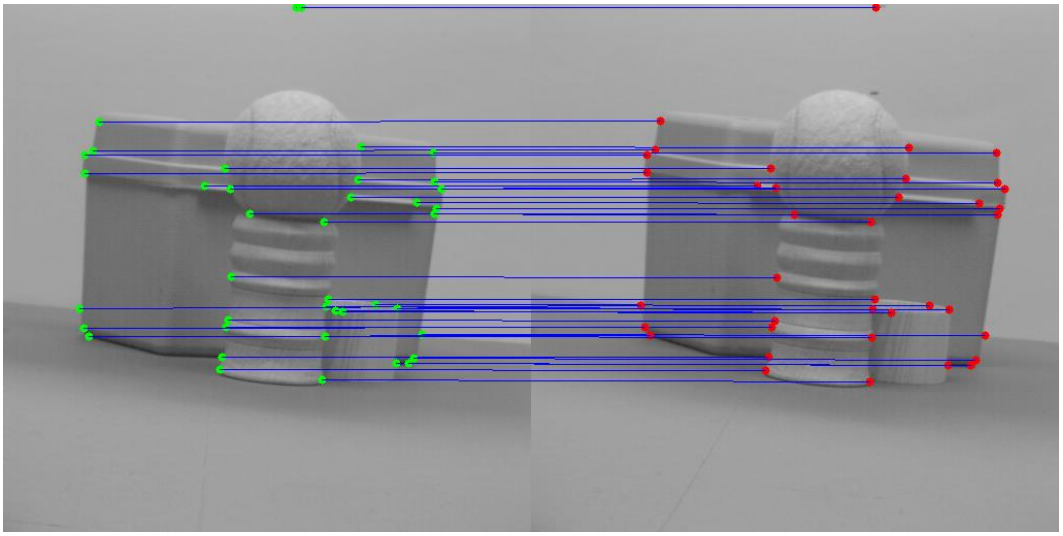
  I have four images for each image:
  Left corners, right corners, relative depth (Note: Since the detected corners and depth aren't seen well in the image here, I have added another image to map corners from the left stereo image to the right stereo image.

  Image1: Balls

  Left Corner Right Corner

Matched features.



Relative Depth

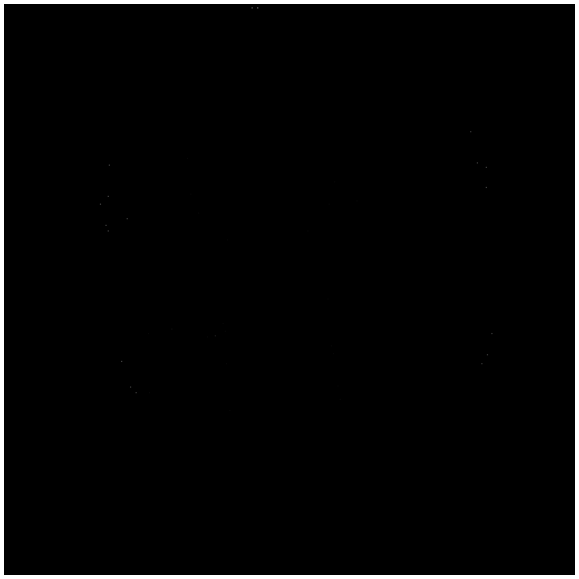
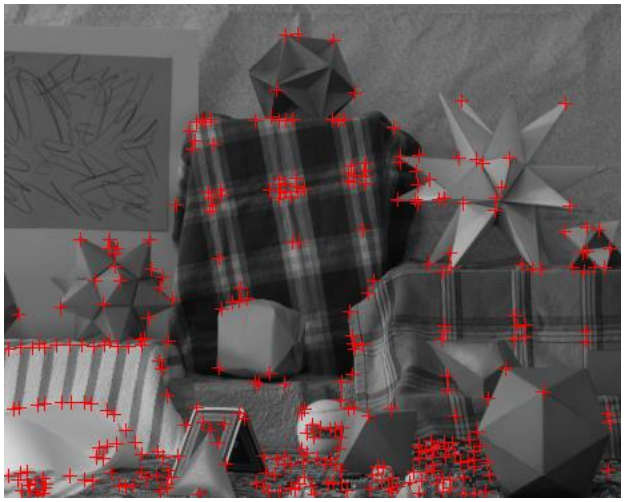


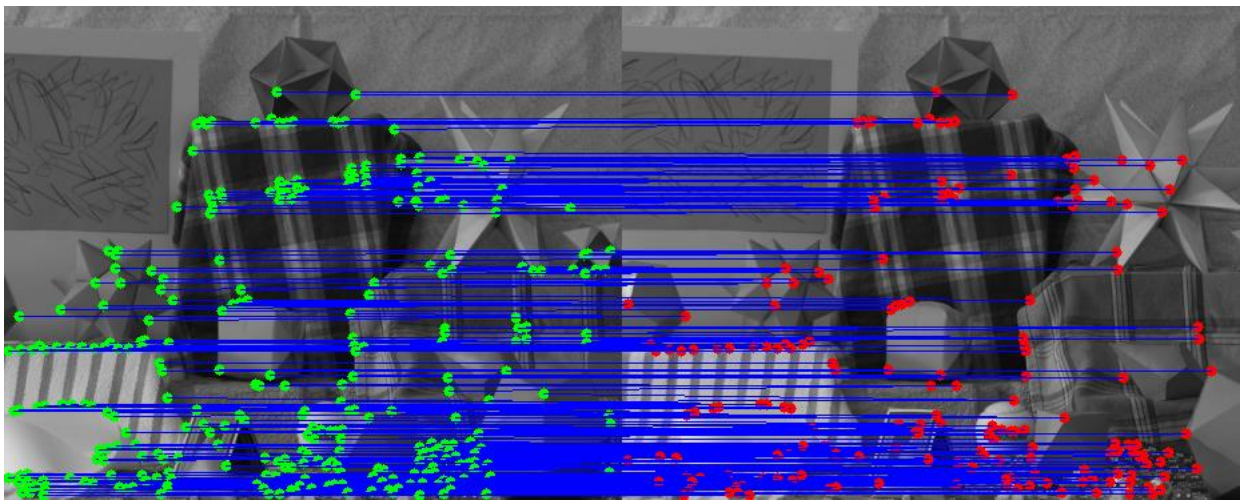
Image2: Moebius

Left Corner:



Right Corner:

Matched Features:



Relative Depth

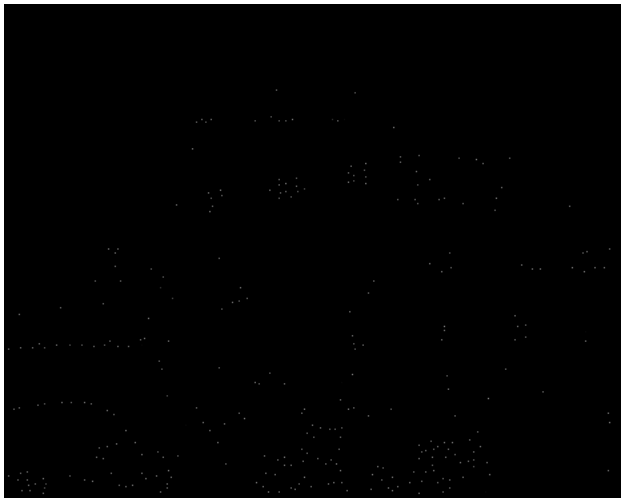
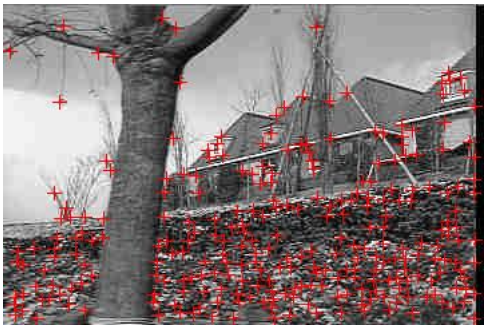


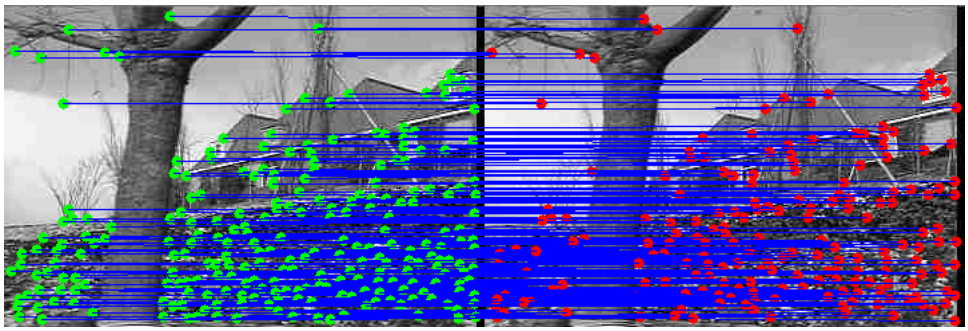
Image 3: Outdoor:

Left Corner

Right Corner



Matched Features:



Relative Depth: